МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»



Лабораторна робота №2

“Моделювання безпілотних систем у Gazebo та ArduPilot SITL. Створення і тестування алгоритмів керування, дослідження динаміки польоту в симуляційному середовищі.”

З дисципліни “Програмно-апаратне забезпечення систем рухомих об’єктів”

Варіант 1

Виконала:

студентка групи ІР-33

Абрам’як Анна

Перевірив:

Шихмат А. О.

Дата виконання: 28.09.2026

Львів – 2026

Мета роботи.

Метою роботи є набуття практичних навичок розгортання та запуску програмного середовища моделювання безпілотної авіаційної системи на основі Gazebo, ArduPilot SITL і ROS 2, автоматизації запуску його компонентів, виконання тестового польоту та розроблення простого алгоритму керування дроном. У результаті роботи студент повинен реалізувати політ дрона до заданої точки з подальшою автоматичною посадкою.

Завдання 1. Підготовка програмного середовища

1. Операційна система

   Розгортання симуляційного середовища виконано в ОС Windows з використанням підсистеми WSL 2. Назва та версія ОС: Ubuntu 22.04.5 LTS. Архітектура: x86_64.

```
anna@DESKTOP-0RLGNKS:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
Codename:       jammy
```

```
anna@DESKTOP-0RLGNKS:~$ uname -m
x86_64
```

2. Перелік використаного програмного забезпечення та його версій

   • Операційна система: Ubuntu 22.04.5 LTS, архітектура x86_64.

   • ROS 2 Humble.

   • ArduPilot SITL: ArduCopter V4.8.0-dev.

   • Gazebo 11.10.2.

   • Python 3.10.12.

   • colcon-core 0.21.3.

3. Перевірка сумісності програмного забезпечення

   Було перевірено сумісність установлених компонентів програмного середовища. ROS 2 Humble використовується в ОС Ubuntu 22.04.5 LTS, Gazebo та ArduPilot SITL успішно запускаються в даному середовищі. Під час запуску симуляції критичних конфліктів між установленими компонентами не виявлено.

4. Інструкція розгортання і запуску симуляційного середовища

   Розгортання середовища моделювання безпілотної авіаційної системи виконано за допомогою командного рядка операційної системи.

   <u>Встановлення ROS 2 Humble:</u>

   sudo apt update && sudo apt install locales -y

   sudo locale-gen en_US en_US.UTF-8

   sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

```
export LANG=en_US.UTF-8
sudo apt install software-properties-common -y
sudo add-apt-repository universe -y
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
sudo apt update
sudo apt install ros-humble-desktop python3-colcon-common-extensions -y
```

Встановлення ArduPilot SITL:

```
sudo apt install git -y
git clone https://github.com/ArduPilot/ardupilot.git
cd ardupilot
git submodule update --init --recursive
Tools/environment_install/install-prereqs-ubuntu.sh -y
source ~/.profile
```

Встановлення Gazebo:

```
sudo apt update
sudo apt install gazebo libgazebo-dev -y
sudo apt install ros-humble-gazebo-ros-pkgs -y
```

Запуск симулятора

```
./start_sim.sh
```

5. Встановлення плагіна зв'язку ardupilot_gazebo та 3D-моделей:

```
cd ~
git clone https://github.com/khancyr/ardupilot_gazebo.git
cd ardupilot_gazebo
mkdir build
cd build
cmake ..
make -j4
sudo make install
```

Для забезпечення взаємодії ROS 2 з ArduPilot було використано робоче середовище ardu_ws, розташоване в каталозі ~/ardu_ws. У цьому середовищі доступний пакет ardupilot_sitl, який

використовується для запуску ArduPilot SITL із DDS-зв'язком. Перед роботою середовище ArduPilot підключається командою:

source ~/ardu_ws/install/setup.bash

Доступність пакета ardupilot_sitl перевірено командою:

ros2 pkg prefix ardupilot_sitl

У результаті отримано шлях /home/anna/ardu_ws/install/ardupilot_sitl.

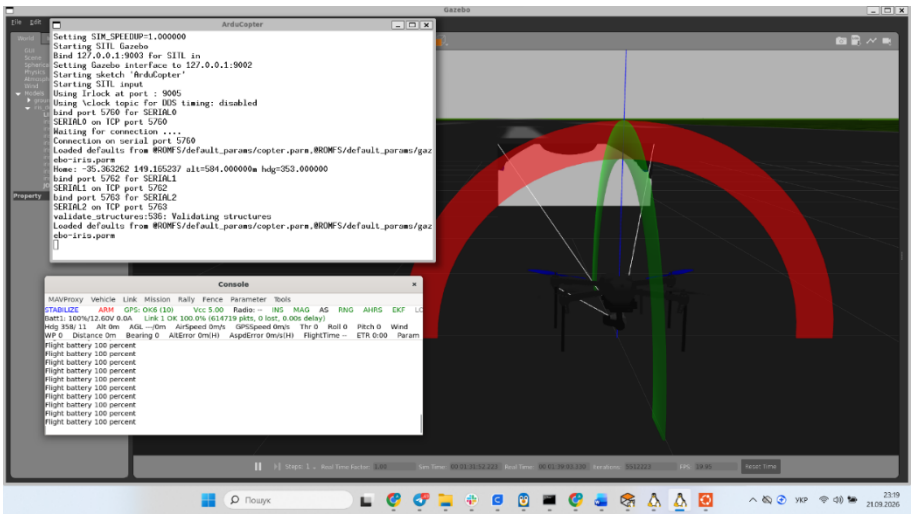
```
anna@DESKTOP-0RLGNKS:~/fpv_labs/src/fpv_lab2/fpv_lab2$ source /opt/ros/humble/setup.bash
source ~/ardu_ws/install/setup.bash
ros2 pkg prefix ardupilot_sitl
/home/anna/ardu_ws/install/ardupilot_sitl
```

6. Перевірка доступності ROS 2 вузлів, тем і сервісів

   Після запуску ArduPilot SITL із DDS-зв'язком було перевірено доступність ROS 2 вузлів, тем і сервісів. У системі доступний вузол /ap, а також теми /ap/status, /ap/pose/filtered і /ap/cmd_vel. Доступні сервіси /ap/prearm_check, /ap/mode_switch, /ap/arm_motors та /ap/experimental/takeoff. Отримані результати підтверджують коректний обмін даними між ROS 2 та ArduPilot SITL.

```
anna@DESKTOP-0RLGNKS:~/ardupilot$ ros2 node list
/ap
anna@DESKTOP-0RLGNKS:~/ardupilot$ ros2 topic list | grep -E '^/ap/(status|pose/filtered|cmd_vel)$'
/ap/cmd_vel
/ap/pose/filtered
/ap/status
anna@DESKTOP-0RLGNKS:~/ardupilot$ ros2 service list | grep -E '^/ap/(prearm_check|mode_switch|arm_motors|experimental/takeoff
/ap/arm_motors
/ap/experimental/takeoff
/ap/mode_switch
/ap/prearm_check
```

7. Перевірити появу моделі дрона у Gazebo



Завдання 2. Автоматизація запуску

Для автоматизації запуску симуляційного середовища в ОС Ubuntu 22.04.5 LTS (x86_64) розроблено Bash-скрипт start_sim.sh. Компоненти системи попередньо встановлено за допомогою apt та git. Скрипт автоматично підключає середовище Gazebo, ROS 2 Humble та робоче середовище ArduPilot, задає шляхи до моделей і плагінів ardupilot_gazebo, запускає Gazebo зі світом iris_arducopter_runway.world, після чого запускає ArduPilot SITL із DDS-зв'язком за допомогою пакета ardupilot_sitl. Запуск середовища виконується командою

./start_sim.sh, а зупинка — комбінацією Ctrl+C. Для завершення залишкових процесів Gazebo у скрипті використовується функція cleanup та обробник сигналу SIGINT.

Особливістю створеного скрипта є використання локальних шляхів до встановлених компонентів у середовищі WSL 2, тому для його запуску необхідна попередньо налаштована структура каталогів ardu_ws, ardupilot_gazebo та fpv_labs.

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ cat ~/start_sim.sh
#!/bin/bash

cleanup() {
    killall -9 gzserver gzclient 2>/dev/null
}

trap cleanup SIGINT

source /usr/share/gazebo/setup.sh
source /opt/ros/humble/setup.bash
source "$HOME/ardu_ws/install/setup.bash"

export GAZEBO_MODEL_PATH=$HOME/ardupilot_gazebo/models:$GAZEBO_MODEL_PATH
export GAZEBO_PLUGIN_PATH=$HOME/ardupilot_gazebo/build:$GAZEBO_PLUGIN_PATH

gazebo --verbose ~/ardupilot_gazebo/worlds/iris_arducopter_runway.world &

sleep 5
cd "$HOME/fpv_labs"

ros2 launch ardupilot_sitl sitl_dds_udp.launch.py \
    transport:=udp4 \
    synthetic_clock:=True \
    wipe:=True \
    model:=gazebo-iris \
    speedup:=1 \
    slave:=0 \
    instance:=0 \
    defaults:=$(ros2 pkg prefix ardupilot_sitl)/share/ardupilot_sitl/config/
default_params/copter.parm,$(ros2 pkg prefix ardupilot_sitl)/share/ardupilot
_sitl/config/default_params/gazebo-iris.parm,$(ros2 pkg prefix ardupilot_sit
l)/share/ardupilot_sitl/config/default_params/dds_udp.parm \
    sim_address:=127.0.0.1 \
    master:=tcp:127.0.0.1:5760 \
    sitl:=127.0.0.1:5501

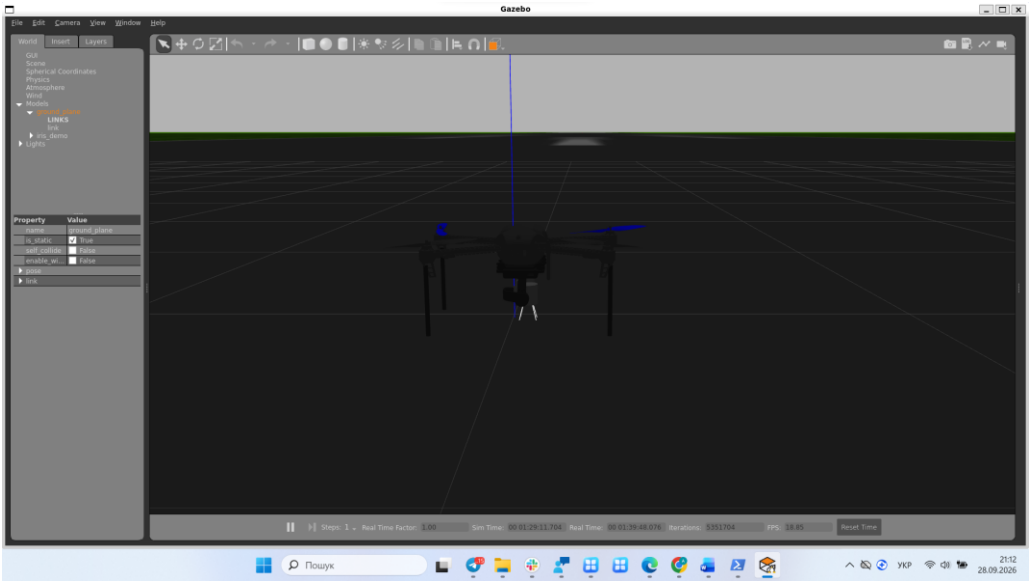
cleanup
```

У скрипті запуск ArduPilot SITL із DDS-зв’язком виконується через sitl_dds_udp.launch.py із використанням конфігурації dds_udp.parm, що забезпечує обмін даних між ArduPilot і ROS2.

Перед першим запуском скрипту було надано право на виконання командою chmod +x ~/start_sim.sh.

Завдання 3. Запуск початкового прикладу

1. Запустити Gazebo, ArduPilot SITL і ROS 2.



Запущено симуляційне середовище Gazebo, ArduPilot SITL та ROS 2 з DDS-зв’язком.

2. Створити робочий простір ROS 2 та перейти до каталогу src.

```
anna@DESKTOP-0RLGNKS:~/fpv_labs$ cd ~/fpv_labs/src
anna@DESKTOP-0RLGNKS:~/fpv_labs/src$ pwd
/home/anna/fpv_labs/src
anna@DESKTOP-0RLGNKS:~/fpv_labs/src$ ls
fpv_lab2
```

Було створено робочий простір fpv_labs та каталог src.

3. Створити Python-пакет ROS 2:

```
anna@DESKTOP-0RLGNKS:~/fpv_labs/src/fpv_lab2$ ls
LICENSE  fpv_lab2  package.xml  resource  setup.cfg  setup.py  test
```

Було створено Python-пакет ROS 2 fpv_lab2 із залежностями rclpy, std_srvs, geometry_msgs, nav_msgs та ardupilot_msgs.

4. Розмістити наданий викладачем програмний код у створеному пакеті відповідно до його структури. Перевірити, що виконуваний вузол доданий до секції console_scripts у файлі setup.py.

```
anna@DESKTOP-0RLGNKS:~/fpv_labs/src/fpv_lab2/fpv_lab2$ find flight_test -type f | grep -v __pycache__ | sort
flight_test/__init__.py
flight_test/flight_state.py
flight_test/flight_test_node.py
flight_test/handlers/__init__.py
flight_test/handlers/arming.py
flight_test/handlers/handler.py
flight_test/handlers/hovering.py
flight_test/handlers/landing.py
flight_test/handlers/prearm_check.py
flight_test/handlers/setting_guided.py
flight_test/handlers/taking_off.py
flight_test/handlers/waiting_for_landing.py
flight_test/handlers/waiting_for_system.py
flight_test/main.py
```

Програмний код було розміщено у пакеті fpv_lab2 відповідно до структури початкового прикладу.

```
anna@DESKTOP-0RLGNKS:~/fpv_labs/src/fpv_lab2/fpv_lab2$ grep -A 4 "console_scripts" ~/fpv_labs/src/fpv_lab2/setup.py
        'console_scripts': [
            'flight_test = fpv_lab2.flight_test.main:main',
        ],
    },
)
```

Виконуваний вузол flight_test додано до секції console_scripts файлу setup.py.

Пакет fpv_lab2 реалізовано як Python-пакет ROS 2. Початковий приклад flight_test містить основний вузол керування польотом та окремі обробники етапів роботи: перевірки готовності системи, переходу в режим GUIDED, увімкнення двигунів, зльоту, зависання та посадки. Файл setup.py містить точку входу для запуску вузла flight_test за допомогою команди ros2 run fpv_lab2 flight_test.

5. Перед складанням пакета варто явно підключити базове середовище ROS 2 та середовище ArduPilot:

source /opt/ros/humble/setup.bash

source /opt/ardu_ws/install/setup.bash

Було підключено базове середовище ROS 2 Humble та середовище ArduPilot. Доступність пакета ardupilot_sitl підтверджено командою ros2 pkg prefix ardupilot_sitl.

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ source /opt/ros/humble/setup.bash
source ~/ardu_ws/install/setup.bash
ros2 pkg prefix ardupilot_sitl
/home/anna/ardu_ws/install/ardupilot_sitl
```

6. Перейти до кореневого каталогу робочого простору та виконати складання пакета:

cd {WORKSPACE_FOLDER}/fpv_labs

colcon build --symlink-install --packages-select fpv_lab2

Пакет fpv_lab2 було успішно складено за допомогою colcon build.

```
anna@DESKTOP-0RLGNKS:~/fpv_labs/src/fpv_lab2/fpv_lab2$ cd ~/fpv_labs
anna@DESKTOP-0RLGNKS:~/fpv_labs$ colcon build --symlink-install --packages-select fpv_lab2
Starting >>> fpv_lab2
/home/anna/.local/lib/python3.10/site-packages/setuptools/_distutils/dist.py:318: UserWarning: Unknow
istribution option: 'tests_require'
  warnings.warn(msg)
--- stderr: fpv_lab2
/home/anna/.local/lib/python3.10/site-packages/setuptools/_distutils/dist.py:318: UserWarning: Unknow
istribution option: 'tests_require'
  warnings.warn(msg)
---
Finished <<< fpv_lab2 [2.40s]

Summary: 1 package finished [2.81s]
  1 package had stderr output: fpv_lab2
```

7. Підключити створене робоче середовище:

source {WORKSPACE_FOLDER}/fpv_labs/install/setup.bash

```
anna@DESKTOP-0RLGNKS:~/fpv_labs$ source ~/fpv_labs/install/setup.bash
anna@DESKTOP-0RLGNKS:~/fpv_labs$ ros2 pkg list | grep fpv_lab2
fpv_lab2
```

Створене робоче середовище fpv_labs було успішно підключено.

8. Перевірити доступність виконуваного вузла:

ros2 pkg executables fpv_lab2

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ ros2 pkg executables fpv_lab2
fpv_lab2 flight_test
fpv_lab2 point_flight
```

Доступність виконуваних вузлів пакета fpv_lab2 було перевірено командою ros2 pkg executables fpv_lab2. У результаті відображено вузли flight_test та point_flight, що підтверджує коректне складання та підключення пакета.

9. В окремому терміналі підключити системне та створене робочі середовища ROS 2. Конкретні шляхи можуть відрізнятися залежно

від способу встановлення:

source /opt/ros/humble/setup.bash

source /opt/ardu_ws/install/setup.bash

source {WORKSPACE_FOLDER}/install/setup.bash

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ source /opt/ros/humble/setup.bash
source ~/ardu_ws/install/setup.bash
source ~/fpv_labs/install/setup.bash
ros2 pkg executables fpv_lab2
fpv_lab2 flight_test
fpv_lab2 point_flight
```

В окремому терміналі було підключено базове середовище ROS 2 Humble, середовище ArduPilot відповідно до фактичного шляху встановлення ~/ardu_ws/install/setup.bash та робоче середовище fpv_labs. Після цього перевірено доступність виконуваних вузлів пакета fpv_lab2.

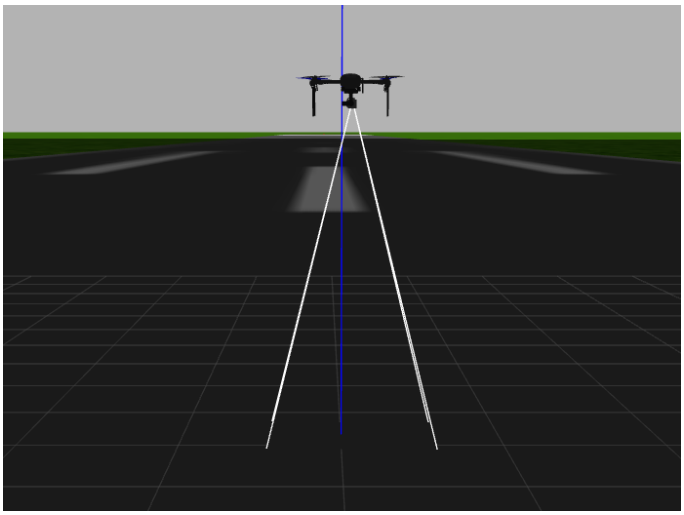
10. Запустити наданий викладачем приклад:

ros2 run fpv_lab2 flight_test

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ ros2 run fpv_lab2 flight_test
[INFO] [1790620805.957166081] [flight_test]: Flight test node started
[INFO] [1790620805.959125119] [flight_test]: Waiting for ArduPilot and ROS services...
[INFO] [1790620806.114194671] [flight_test]: ArduPilot and all services are available
[INFO] [1790620806.314502807] [flight_test]: Running pre-arm check
[INFO] [1790620806.321066358] [flight_test]: Pre-arm check passed: Vehicle is Armable
[INFO] [1790620806.513062822] [flight_test]: Requesting flight mode 4
[INFO] [1790620806.520193429] [flight_test]: GUIDED mode enabled
[INFO] [1790620806.714415444] [flight_test]: Requesting motor arming
[INFO] [1790620806.719492473] [flight_test]: Motors armed
[INFO] [1790620806.913136877] [flight_test]: Requesting takeoff to 2.0 m
[INFO] [1790620806.921884031] [flight_test]: Takeoff command accepted
[INFO] [1790620806.923179017] [flight_test]: Hovering for 30.0 seconds
```

Початковий приклад flight_test було запущено командою ros2 run fpv_lab2 flight_test.

11. Переконатися, що дрон виконує автоматичний зліт.

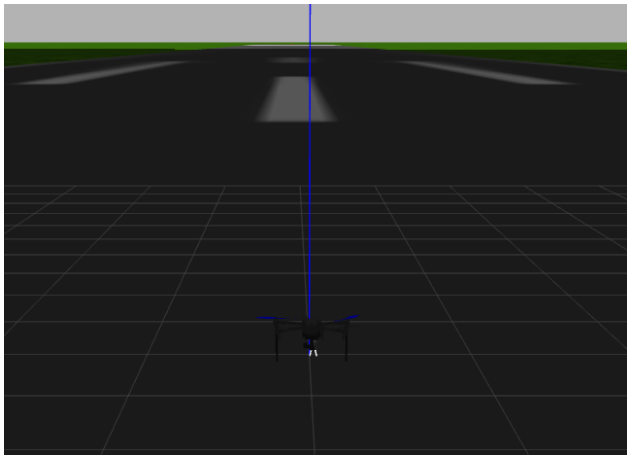


Дрон успішно виконав автоматичний зліт на висоту 2 м.

12. Перевірити утримання висоти в режимі зависання.

Дрон утримував висоту 2 м у режимі зависання протягом 30 секунд.

13. Дочекатися автоматичної посадки.



Після завершення зависання дрон автоматично перейшов у режим посадки та успішно приземлився.

14. Зберегти журнал виконання програми або його основну частину для звіту.

```
anna@DESKTOP-0RLGNKS:~$ ros2 run fpv_lab2 flight_test
[INFO] [1790621215.022660938] [flight_test]: Flight test node started
[INFO] [1790621215.025325478] [flight_test]: Waiting for ArduPilot and ROS services...
[INFO] [1790621215.359692218] [flight_test]: ArduPilot and all services are available
[INFO] [1790621215.559852758] [flight_test]: Running pre-arm check
[INFO] [1790621215.566735508] [flight_test]: Pre-arm check passed: Vehicle is Armable
[INFO] [1790621215.759939048] [flight_test]: Requesting flight mode 4
[INFO] [1790621215.768144618] [flight_test]: GUIDED mode enabled
[INFO] [1790621215.960881238] [flight_test]: Requesting motor arming
[INFO] [1790621215.974424978] [flight_test]: Motors armed
[INFO] [1790621216.160093178] [flight_test]: Requesting takeoff to 2.0 m
[INFO] [1790621216.167551838] [flight_test]: Takeoff command accepted
[INFO] [1790621216.168342218] [flight_test]: Hovering for 30.0 seconds
[INFO] [1790621249.360069488] [flight_test]: Hover completed, starting landing
[INFO] [1790621249.559938068] [flight_test]: Requesting flight mode 9
[INFO] [1790621249.567377828] [flight_test]: LAND mode enabled
[INFO] [1790621267.159661244] [flight_test]: Landing completed
[INFO] [1790621267.160229924] [flight_test]: Flight test completed successfully
anna@DESKTOP-0RLGNKS:~$
```

Основну частину журналу виконання програми було збережено. Журнал підтверджує успішне виконання автоматичного зльоту, зависання протягом 30 секунд та автоматичної посадки.

Завдання 4. Розроблення власного алгоритму

На основі наданого прикладу додати власний програмний модуль, який забезпечить політ до координат, заданих викладачем.

Алгоритм повинен:

1. Виконати перевірку готовності системи.
2. Перевести дрон у необхідний режим.
3. Перевести двигуни в режим arm.
4. Виконати зліт на задану висоту.
5. Отримувати поточне положення дрона.
6. Обчислювати відстань і напрямок до цільової точки.

7. Формувати команди горизонтального руху.

8. Зменшувати швидкість під час наближення до цілі або іншим способом забезпечувати точне позиціонування.

9. Зафіксувати досягнення точки з установленою похибкою.

10.Зупинити горизонтальний рух.

11.Виконати автоматичну посадку.

12.Коректно завершити роботу програми.

Для варіанта 1 задано зміщення Δx = 7,0 м та Δy = 1,4 м. Цільова точка визначається відносно фактичного початкового положення дрона за формулами x_target = x_0 − Δx та y_target = y_0 − Δy.

Власний програмний модуль реалізовано у файлі target_flight.py. У файлі setup.py його зареєстровано як виконуваний вузол point_flight, тому запуск алгоритму виконується командою ros2 run fpv_lab2 point_flight.

Усі етапи алгоритму було успішно реалізовано та перевірено під час моделювання. Програма виконала перевірку готовності системи, переведення дрона в режим GUIDED, переведення двигунів у режим arm та автоматичний зліт на задану висоту 2 м. Після досягнення висоти 1,91 м дрон стабілізувався протягом 3 секунд, після чого розпочав горизонтальний політ. Під час польоту програма отримувала поточні координати, обчислювала відстань і напрямок до заданої точки, формувала команди горизонтального руху та поступово зменшувала швидкість під час наближення до цілі. Дрон досяг цільової точки з похибкою 0,234 м, після чого горизонтальний рух було зупинено, виконано автоматичну посадку та коректно завершено роботу програми. Отримана похибка відповідає рекомендованому в методичних вказівках діапазону 0,2–0,3 м.

```
anna@DESKTOP-0RLGNKS:/mnt/c/Users/Anna$ ros2 run fpv_lab2 point_flight
[INFO] [1791756213.091884102] [target_flight]: Target flight node started
[INFO] [1791756213.092831069] [target_flight]: Waiting for ArduPilot, pose and services...
[INFO] [1791756213.095443753] [target_flight]: Start: x=0.018, y=-0.011
[INFO] [1791756213.096208885] [target_flight]: Target: x=-6.982, y=-1.411
[INFO] [1791756213.159510462] [target_flight]: System ready
[INFO] [1791756213.259815695] [target_flight]: Running pre-arm check
[INFO] [1791756213.266300679] [target_flight]: Pre-arm check passed
[INFO] [1791756213.359956108] [target_flight]: Requesting flight mode 4
[INFO] [1791756213.363924692] [target_flight]: GUIDED mode enabled
[INFO] [1791756213.460069273] [target_flight]: Requesting motor arming
[INFO] [1791756213.473416943] [target_flight]: Motors armed
[INFO] [1791756213.559667622] [target_flight]: Requesting takeoff to 2.0 m
[INFO] [1791756213.566712344] [target_flight]: Takeoff command accepted
[INFO] [1791756219.059746134] [target_flight]: Takeoff altitude reached: 1.91 m
[INFO] [1791756219.060374477] [target_flight]: Stabilizing at target altitude
[INFO] [1791756222.159998718] [target_flight]: Stabilization completed after 3.0 seconds
[INFO] [1791756222.160672112] [target_flight]: Starting horizontal flight
[INFO] [1791756222.259689130] [target_flight]: Position: x=-0.04, y=0.03, distance=7.09 m
[INFO] [1791756223.259798107] [target_flight]: Position: x=-0.36, y=-0.02, distance=6.76 m
[INFO] [1791756224.359778211] [target_flight]: Position: x=-1.49, y=-0.28, distance=5.61 m
[INFO] [1791756225.359841213] [target_flight]: Position: x=-2.44, y=-0.47, distance=4.64 m
[INFO] [1791756226.459677245] [target_flight]: Position: x=-3.48, y=-0.67, distance=3.58 m
[INFO] [1791756227.459977288] [target_flight]: Position: x=-4.45, y=-0.87, distance=2.59 m
[INFO] [1791756228.560518608] [target_flight]: Position: x=-5.53, y=-1.10, distance=1.49 m
[INFO] [1791756229.660472656] [target_flight]: Position: x=-6.34, y=-1.27, distance=0.66 m
[INFO] [1791756230.660350820] [target_flight]: Position: x=-6.66, y=-1.34, distance=0.33 m
[INFO] [1791756231.159605793] [target_flight]: Target reached
[INFO] [1791756231.160034016] [target_flight]: Actual: x=-6.754, y=-1.358
[INFO] [1791756231.160412221] [target_flight]: Position error: 0.234 m
[INFO] [1791756232.260620954] [target_flight]: Requesting flight mode 9
[INFO] [1791756232.268903978] [target_flight]: LAND mode enabled
[INFO] [1791756248.159778418] [target_flight]: Landing completed
[INFO] [1791756248.160539280] [target_flight]: Target flight completed successfully
```

Завдання 5. Перевірка результату

Під час виконання польоту необхідно зафіксувати:

• початкове положення дрона;

• задані координати цілі;

• фактичні координати перед початком посадки;

• похибку досягнення цільової точки;

• факт успішного завершення посадки;

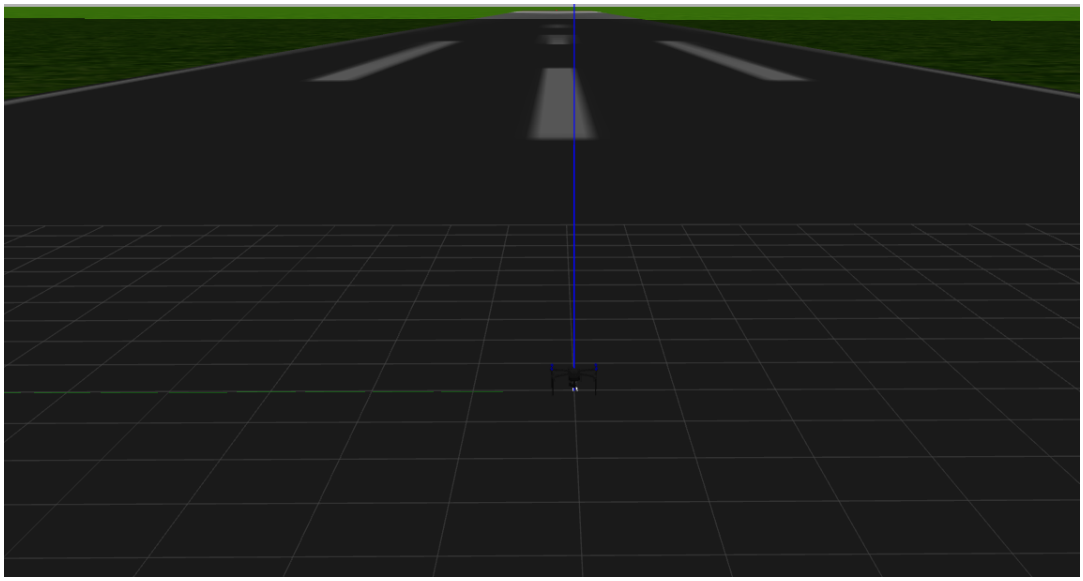
• повідомлення програми про основні етапи польоту.

Необхідно зробити щонайменше два знімки екрана:

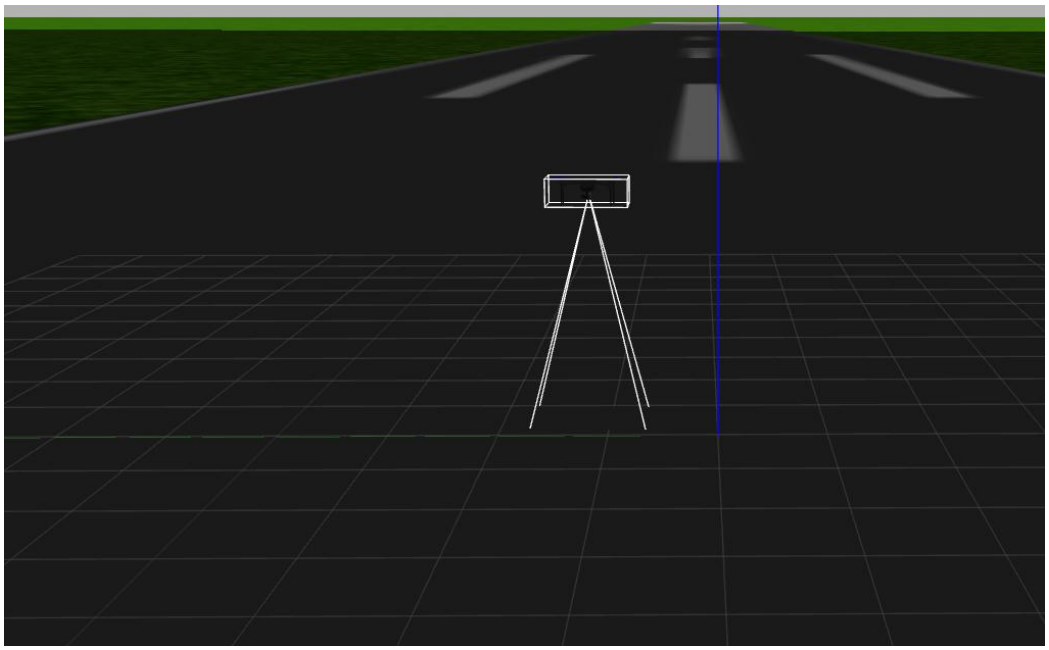
• модель дрона на початку маршруту;

• модель дрона в кінцевій точці перед посадкою або після завершення посадки.

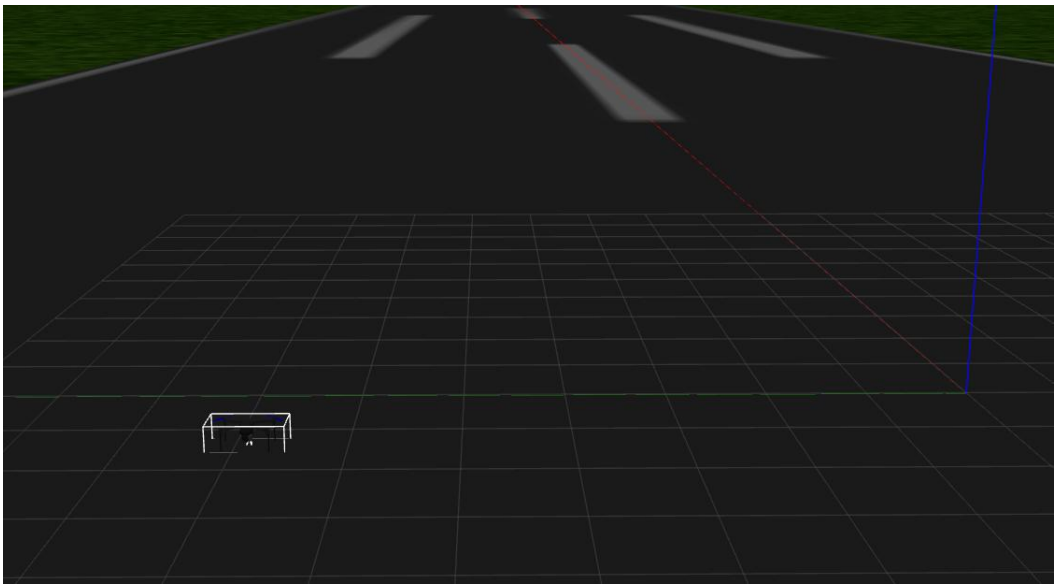
Під час виконання польоту було зафіксовано початкове положення дрона: x = 0,018 м, y = -0,011 м. Задана цільова точка: x = -6,982 м, y = -1,411 м. Перед початком посадки фактичне положення дрона становило: x = -6,754 м, y = -1,358 м. Похибка досягнення цільової точки становила 0,234 м. Після досягнення цілі дрон успішно виконав автоматичну посадку, а програма завершила роботу без помилок. Основні етапи польоту було зафіксовано в журналі виконання програми.



Початкове положення дрона перед виконанням маршруту.



Рух дрона до заданої цільової точки.



Положення дрона в кінцевій точці після завершення автоматичної посадки.

Похибка досягнення цільової точки визначається за формулою:

$$e = \sqrt{(x_{actual} - x_{target})^2 + (y_{actual} - y_{target})^2}$$

$$e = \sqrt{\left\{\left(-6{,}754 - (-6{,}982)\right)^2 + \left(-1{,}358 - (-1{,}411)\right)^2\right\}}$$

$e = 0{,}234$ м

Отримана похибка позиціонування становить 0,234 м, що відповідає рекомендованому допустимому діапазону 0,2–0,3 м.

Програмний код модуля target_flight.py для реалізації польоту дрона до заданої точки

```
#!/usr/bin/env python3

import math
from enum import Enum, auto
from functools import partial

import rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data

from geometry_msgs.msg import PoseStamped, TwistStamped
from std_srvs.srv import Trigger
from ardupilot_msgs.msg import Status
from ardupilot_msgs.srv import ArmMotors, ModeSwitch, Takeoff


class State(Enum):
    WAITING = auto()
    PREARM = auto()
    GUIDED = auto()
    ARMING = auto()
    TAKEOFF = auto()
    WAIT_ALTITUDE = auto()
    STABILIZING = auto()
    MOVE_TO_TARGET = auto()
    STOPPING = auto()
    LANDING = auto()
    WAIT_LANDING = auto()
    FINISHED = auto()
    ERROR = auto()


class TargetFlightNode(Node):
    GUIDED_MODE = 4
    LAND_MODE = 9

    TAKEOFF_ALTITUDE = 2.0
    STABILIZATION_DURATION = 3.0

    DELTA_X = 7.0
    DELTA_Y = 1.4

    TARGET_TOLERANCE = 0.25
    MAX_SPEED = 1.0
    KP = 0.5

    def __init__(self):
        super().__init__("target_flight")

        self.state = State.WAITING
        self.status = None
        self.pose = None

        self.initial_x = None
        self.initial_y = None
        self.target_x = None
        self.target_y = None

        self.request_in_progress = False
        self.stop_counter = 0
        self.last_log_time = 0.0
        self.stabilization_started_at = None
```

```
        self.status_sub = self.create_subscription(
            Status, "/ap/status", self.status_callback, 10
        )

        self.pose_sub = self.create_subscription(
            PoseStamped, "/ap/pose/filtered", self.pose_callback, qos_profile_sensor_data
        )

        self.velocity_pub = self.create_publisher(
            TwistStamped, "/ap/cmd_vel", qos_profile_sensor_data
        )

        self.prearm_client = self.create_client(
            Trigger, "/ap/prearm_check"
        )

        self.mode_client = self.create_client(
            ModeSwitch, "/ap/mode_switch"
        )

        self.arm_client = self.create_client(
            ArmMotors, "/ap/arm_motors"
        )

        self.takeoff_client = self.create_client(
            Takeoff, "/ap/experimental/takeoff"
        )

        self.timer = self.create_timer(0.1, self.control_loop)

        self.get_logger().info("Target flight node started")
        self.get_logger().info("Waiting for ArduPilot, pose and services...")

    def status_callback(self, msg):
        self.status = msg

    def pose_callback(self, msg):
        self.pose = msg

        if self.initial_x is None:
            self.initial_x = msg.pose.position.x
            self.initial_y = msg.pose.position.y

            self.target_x = self.initial_x - self.DELTA_X
            self.target_y = self.initial_y - self.DELTA_Y

            self.get_logger().info(
                f"Start: x={self.initial_x:.3f}, y={self.initial_y:.3f}"
            )
            self.get_logger().info(
                f"Target: x={self.target_x:.3f}, y={self.target_y:.3f}"
            )

    def services_ready(self):
        return (
            self.prearm_client.service_is_ready()
            and self.mode_client.service_is_ready()
            and self.arm_client.service_is_ready()
            and self.takeoff_client.service_is_ready()
        )

    def control_loop(self):
        if self.state == State.WAITING:
            if (
                self.status is not None
                and self.pose is not None
                and self.services_ready()
```

```
        ):
            self.get_logger().info("System ready")
            self.state = State.PREARM

    elif self.state == State.PREARM:
        if not self.request_in_progress:
            self.request_in_progress = True
            future = self.prearm_client.call_async(Trigger.Request())
            future.add_done_callback(self.prearm_callback)
            self.get_logger().info("Running pre-arm check")

    elif self.state == State.GUIDED:
        if not self.request_in_progress:
            self.call_mode(self.GUIDED_MODE, self.guided_callback)

    elif self.state == State.ARMING:
        if not self.request_in_progress:
            self.request_in_progress = True
            request = ArmMotors.Request()
            request.arm = True
            future = self.arm_client.call_async(request)
            future.add_done_callback(self.arm_callback)
            self.get_logger().info("Requesting motor arming")

    elif self.state == State.TAKEOFF:
        if not self.request_in_progress:
            self.request_in_progress = True
            request = Takeoff.Request()
            request.alt = float(self.TAKEOFF_ALTITUDE)
            future = self.takeoff_client.call_async(request)
            future.add_done_callback(self.takeoff_callback)
            self.get_logger().info(
                f"Requesting takeoff to {self.TAKEOFF_ALTITUDE:.1f} m"
            )

    elif self.state == State.WAIT_ALTITUDE:
        if self.pose is not None:
            altitude = self.pose.pose.position.z

            if altitude >= 1.9:
                self.get_logger().info(
                    f"Takeoff altitude reached: {altitude:.2f} m"
                )
                self.get_logger().info("Stabilizing at target altitude")
                self.stabilization_started_at = (
                    self.get_clock().now().nanoseconds / 1e9
                )
                self.state = State.STABILIZING

    elif self.state == State.STABILIZING:
        self.publish_velocity(0.0, 0.0)
        now = self.get_clock().now().nanoseconds / 1e9

        if (
            self.stabilization_started_at is not None
            and now - self.stabilization_started_at
            >= self.STABILIZATION_DURATION
        ):
            self.get_logger().info(
                f"Stabilization completed after "
                f"{self.STABILIZATION_DURATION:.1f} seconds"
            )
            self.get_logger().info("Starting horizontal flight")
            self.state = State.MOVE_TO_TARGET

    elif self.state == State.MOVE_TO_TARGET:
        self.move_to_target()
```

```
        elif self.state == State.STOPPING:
            self.publish_velocity(0.0, 0.0)

            self.stop_counter += 1

            if self.stop_counter >= 10:
                self.state = State.LANDING

        elif self.state == State.LANDING:
            if not self.request_in_progress:
                self.call_mode(self.LAND_MODE, self.land_callback)

        elif self.state == State.WAIT_LANDING:
            if self.status is not None and not self.status.flying:
                self.get_logger().info("Landing completed")
                self.get_logger().info("Target flight completed successfully")
                self.state = State.FINISHED

    def move_to_target(self):
        if self.pose is None:
            return

        x = self.pose.pose.position.x
        y = self.pose.pose.position.y

        error_x = self.target_x - x
        error_y = self.target_y - y

        distance = math.hypot(error_x, error_y)

        if distance <= self.TARGET_TOLERANCE:
            self.publish_velocity(0.0, 0.0)

            self.get_logger().info("Target reached")
            self.get_logger().info(
                f"Actual: x={x:.3f}, y={y:.3f}"
            )
            self.get_logger().info(
                f"Position error: {distance:.3f} m"
            )

            self.stop_counter = 0
            self.state = State.STOPPING
            return

        speed = min(self.MAX_SPEED, self.KP * distance)

        velocity_x = speed * error_x / distance
        velocity_y = speed * error_y / distance

        self.publish_velocity(velocity_x, velocity_y)

        now = self.get_clock().now().nanoseconds / 1e9

        if now - self.last_log_time >= 1.0:
            self.get_logger().info(
                f"Position: x={x:.2f}, y={y:.2f}, "
                f"distance={distance:.2f} m"
            )
            self.last_log_time = now

    def publish_velocity(self, vx, vy):
        msg = TwistStamped()

        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = "map"
```

```
        msg.twist.linear.x = float(vx)
        msg.twist.linear.y = float(vy)
        msg.twist.linear.z = 0.0

        msg.twist.angular.x = 0.0
        msg.twist.angular.y = 0.0
        msg.twist.angular.z = 0.0

        self.velocity_pub.publish(msg)

    def call_mode(self, mode, callback):
        self.request_in_progress = True

        request = ModeSwitch.Request()
        request.mode = mode

        future = self.mode_client.call_async(request)
        future.add_done_callback(callback)

        self.get_logger().info(f"Requesting flight mode {mode}")

    def prearm_callback(self, future):
        self.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            self.fail(f"Pre-arm service failed: {error}")
            return

        if not response.success:
            self.fail(f"Vehicle is not armable: {response.message}")
            return

        self.get_logger().info("Pre-arm check passed")
        self.state = State.GUIDED

    def guided_callback(self, future):
        self.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            self.fail(f"GUIDED request failed: {error}")
            return

        if not response.status:
            self.fail("Could not enable GUIDED mode")
            return

        self.get_logger().info("GUIDED mode enabled")
        self.state = State.ARMING

    def arm_callback(self, future):
        self.request_in_progress = False

        try:
            response = future.result()
        except Exception as error:
            self.fail(f"Arming failed: {error}")
            return

        if not response.result:
            self.fail("ArduPilot rejected arming")
            return
```

```
            self.get_logger().info("Motors armed")
            self.state = State.TAKEOFF

        def takeoff_callback(self, future):
            self.request_in_progress = False

            try:
                response = future.result()
            except Exception as error:
                self.fail(f"Takeoff failed: {error}")
                return

            if not response.status:
                self.fail("ArduPilot rejected takeoff")
                return

            self.get_logger().info("Takeoff command accepted")
            self.state = State.WAIT_ALTITUDE

        def land_callback(self, future):
            self.request_in_progress = False

            try:
                response = future.result()
            except Exception as error:
                self.fail(f"LAND request failed: {error}")
                return

            if not response.status:
                self.fail("Could not enable LAND mode")
                return

            self.get_logger().info("LAND mode enabled")
            self.state = State.WAIT_LANDING

        def fail(self, message):
            self.get_logger().error(message)
            self.publish_velocity(0.0, 0.0)
            self.state = State.ERROR


    def main(args=None):
        rclpy.init(args=args)

        node = TargetFlightNode()

        try:
            while (
                rclpy.ok()
                and node.state != State.FINISHED
                and node.state != State.ERROR
            ):
                rclpy.spin_once(node, timeout_sec=0.1)

        except KeyboardInterrupt:
            pass

        finally:
            node.publish_velocity(0.0, 0.0)
            node.destroy_node()

            if rclpy.ok():
                rclpy.shutdown()


    if __name__ == "__main__":
        main()
```

Репозиторій програмного коду
Посилання на GitHub-репозиторій: https://github.com/AbramiakA/FPV/tree/lab2

Висновок

У ході лабораторної роботи було розгорнуто та налаштовано симуляційне середовище на основі Gazebo, ArduPilot SITL і ROS 2. Було перевірено роботу початкового прикладу автоматичного зльоту, зависання та посадки дрона. На основі наданого прикладу розроблено власний алгоритм польоту до заданої точки. Дрон успішно виконав зліт, стабілізацію в режимі зависання, горизонтальний політ до цільових координат, зменшення швидкості під час наближення до цілі та автоматичну посадку. Похибка позиціонування становила 0,234 м, що відповідає рекомендованому допустимому діапазону 0,2–0,3 м. Програму було завершено без критичних помилок, а результати моделювання підтвердили коректність роботи розробленого алгоритму.